

DIGITAL FILTER IMPLEMENTATION

Low-Pass Digital Filter

$F_c = 100\text{Hz}$

Julian Jaramillo & Hugo Bueno

Digital Signal Processing

May 2024

Aplicación Objetivo: Diferencial Electrónico

El equipo UPM Racing¹ participa anualmente en las competiciones de Formula Student, en la que se realizan varias pruebas con el objetivo de ser el equipo más rápido en completarlas. Una de las pruebas más importantes es el *Skidpad*, la cual consiste en realizar un circuito con forma de infinito, dando una vuelta circular en cada dirección. La línea verde de la Ilustración 1 muestra el ejemplo de una trayectoria más realista.

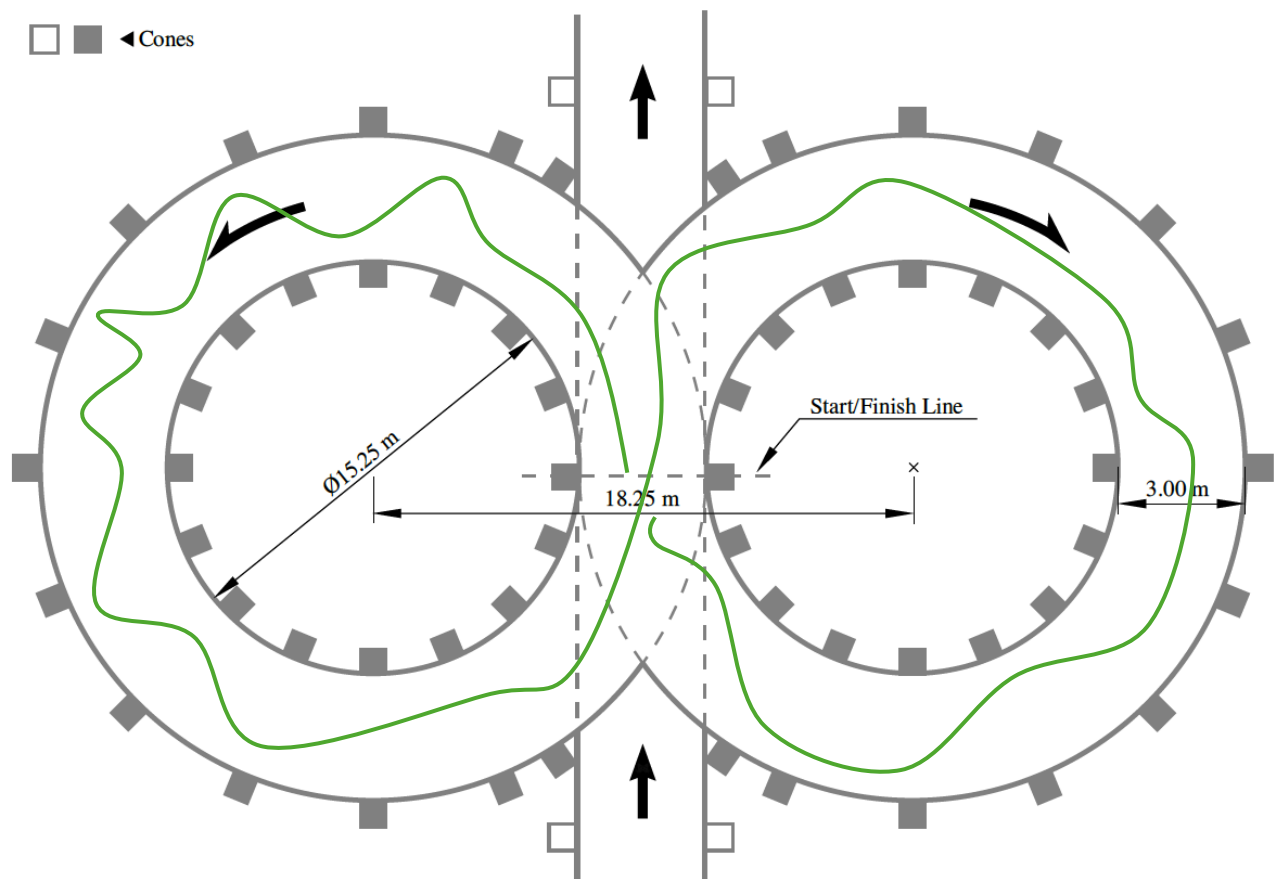


Ilustración 1: Circuito Skidpad de Formula Student

El equipo de nuestra Universidad tiene como característica especial **su diferencial electrónico**. Este sistema comienza con una PCB (Front Signals, Ilustración 2) que toma las señales de intensidad del acelerador y de giro del volante, realiza los cálculos correspondientes en la ECU y manda las distintas potencias eléctricas a los motores de cada rueda a través de los inversores. **No obstante, debido a las vibraciones que sufre el volante durante el rodaje (Ilustración 3) y a la distancia entre el volante y Front Signals (Ilustración 4), la señal de giro suele contener mucho ruido y se filtra al entrar a dicha PCB.**

¹ <https://upm'racing.es/>

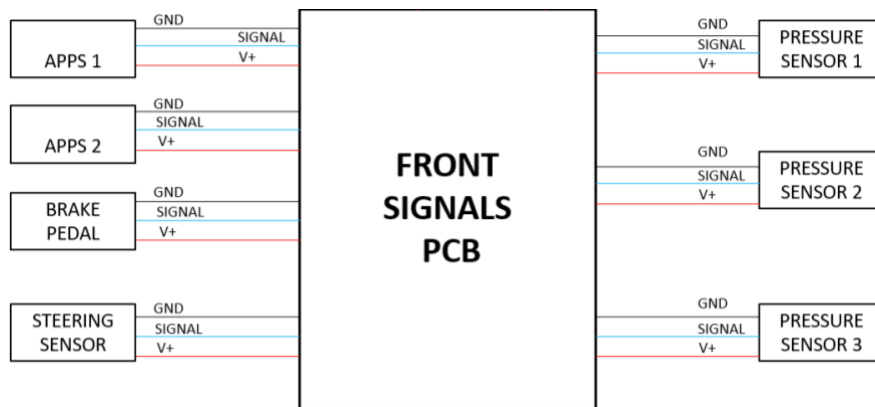


Ilustración 2: Diagrama de bloques de la PCB de Front Signals



Ilustración 3: Primera causa de ruido - vibración del volante

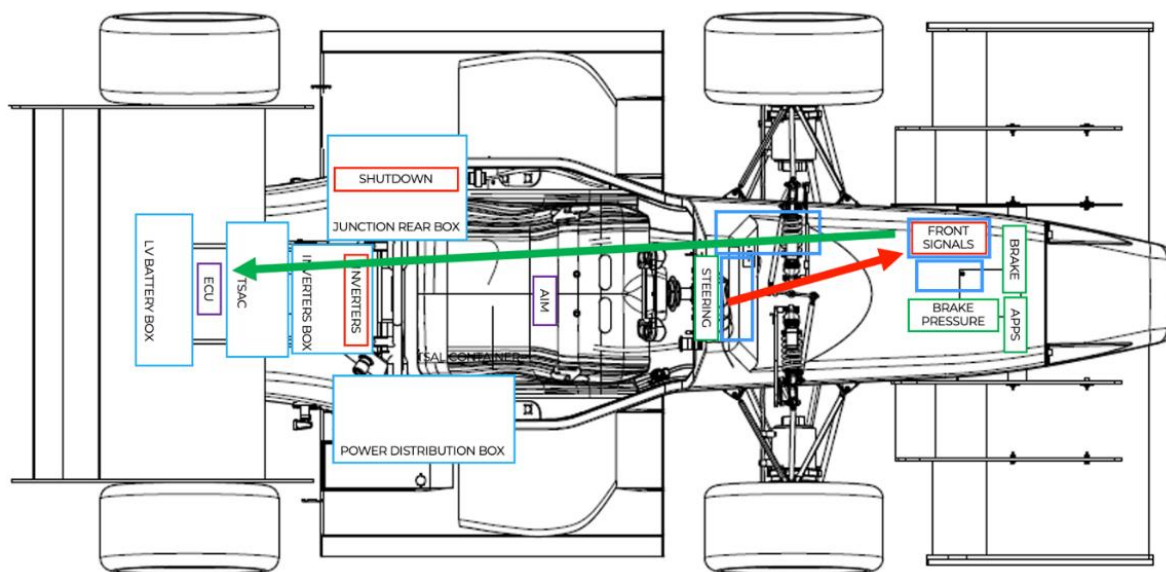


Ilustración 4: Segunda causa de ruido - Posición en el coche

Considerando que el piloto está constantemente corrigiendo la dirección al maniobrar el volante mientras sigue la trayectoria real del Skidpad, y las principales causas de ruido que afectan a la lectura de esta señal. Se diseña un **filtro paso bajo de frecuencia de corte a 100Hz** con una frecuencia de muestreo de 10 kHz; sin embargo, como la FPGA impone una frecuencia de muestreo de 100kHz, **se escala de tal manera que funcionará con una frecuencia de corte de 1000Hz**.

Matlab: Simulink + fdatool

Características de nuestro filtro:

- $f_{\text{cut-off}} = 1000\text{Hz}$
- $f_{\text{stop}} = 3000\text{Hz}$
- Atenuación = 80dB
- $f_s = 100\text{kHz}$
- Orden = 127

Las características del filtro, así como el ancho de palabra con el que se exporta el mismo, se pueden ver en las Ilustraciones 5 y 6, que muestran la fdatool de Matlab.

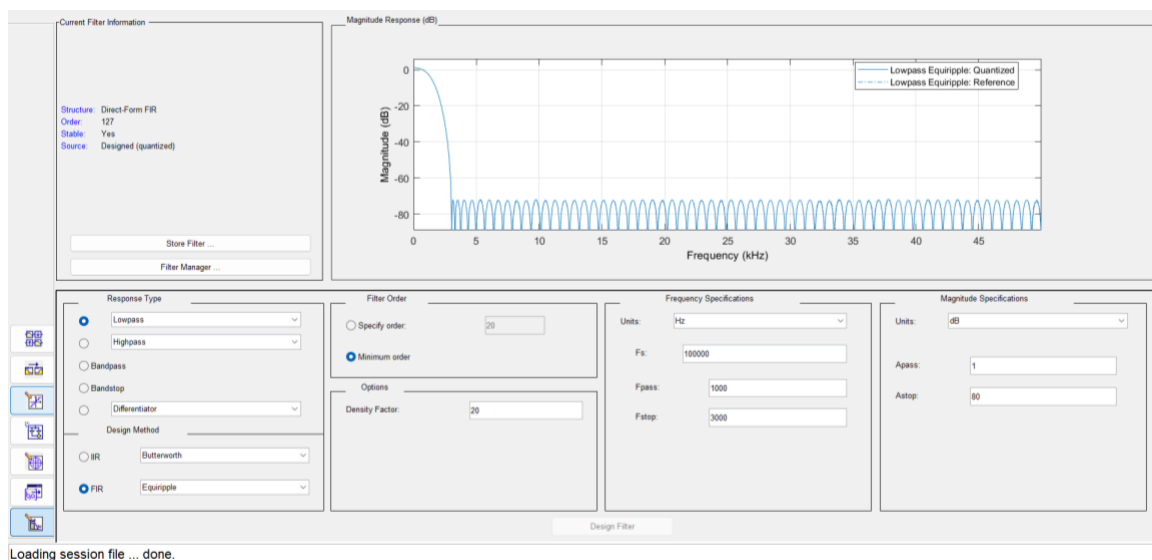


Ilustración 5: Diseño del filtro en fdatool

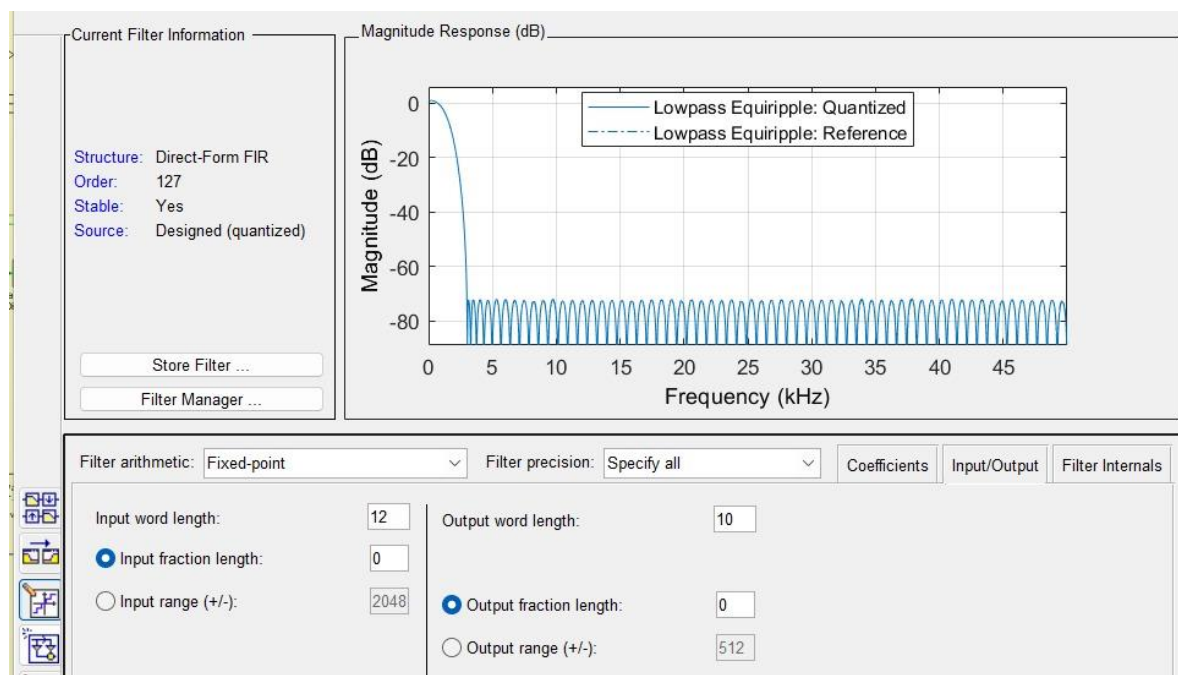


Ilustración 6: Especificaciones de word length en Quantization

Una vez la fdatool genera el bloque del filtro, construimos un modelo de Simulink que integre la señal objetivo a filtrar. Mediante la combinación de señales de varias frecuencia (entre las que se introduce ruido) y bloques sumadores, de selección y scopes, se tiene el sistema mostrado en la Ilustración 7. En la ilustración 8 se observan tanto la señal de entrada como la respuesta del filtro.

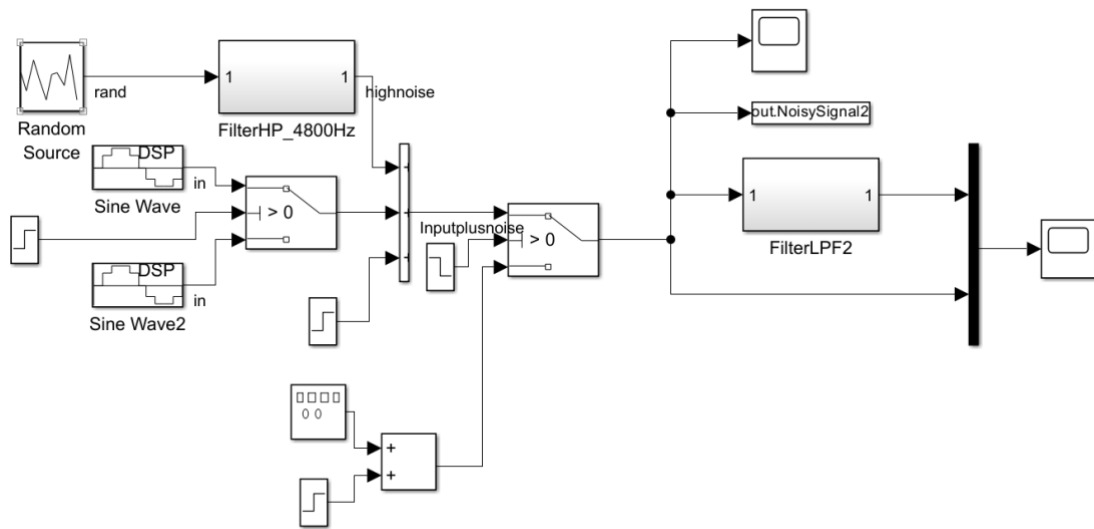


Ilustración 7: Modelo de Simulink

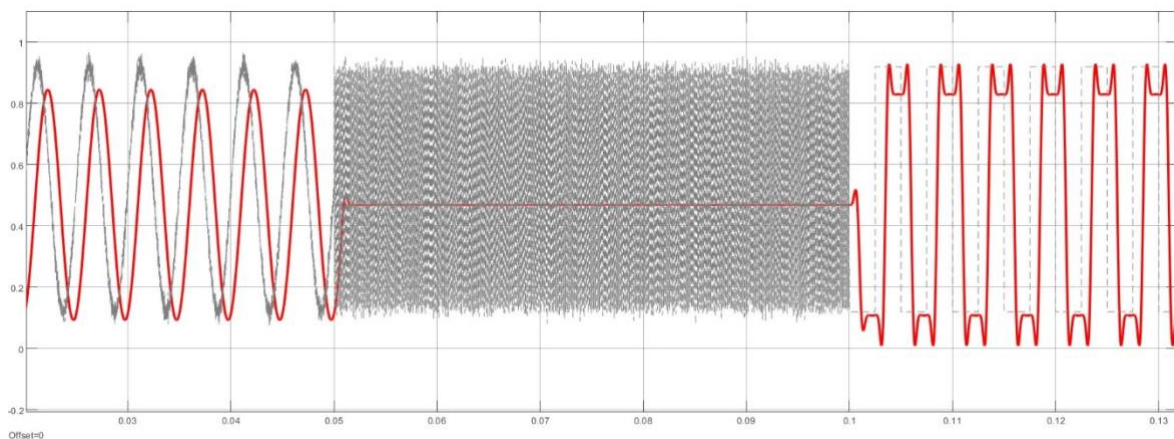


Ilustración 8: Respuesta del filtro en Matlab (en rojo) ante distintas entradas (de izquierda a derecha: seno, ruido y cuadrada)

Vivado

Se crea una señal en Simulink configurada de tal manera que genere formas de onda senoidales en alta y baja frecuencia y después forma de onda cuadrada para validar el funcionamiento del filtro; esta señal se exporta al Workspace de Matlab mediante el bloque “out.NoisySignal2”. Al ser una suma de señales que se presentan en diferentes intervalos de tiempo, se la debe pasar a una variable de dos dimensiones $[x(1,:) = y(1,1,:);]$ y hacer su transpuesta para poder ser cargada como estímulo en Vivado.

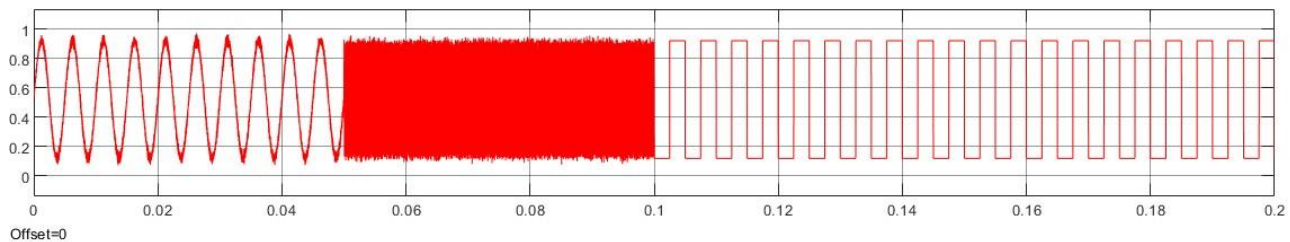


Ilustración 9: Señal de entrada al filtro en Vivado (de izquierda a derecha: seno, ruido y cuadrada)

Después de validar que la señal $[x]$ cumple con los límites de amplitud $[0;1]$, se procede a generar el archivo HDL para poder ser simulada en Vivado, definiendo como estímulo la entrada $[x]$ (Ilustración 10). Y la arquitectura del filtro.

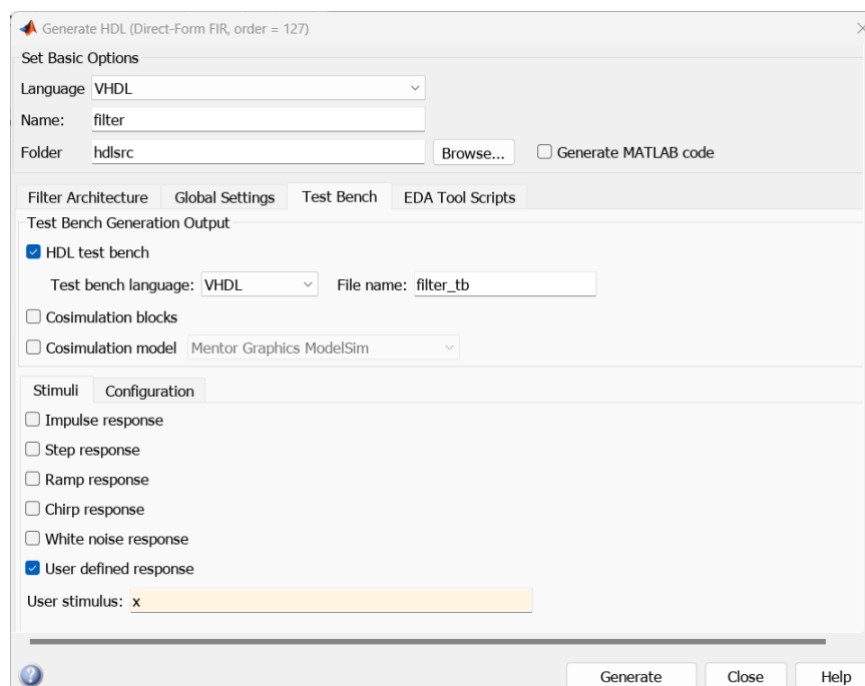
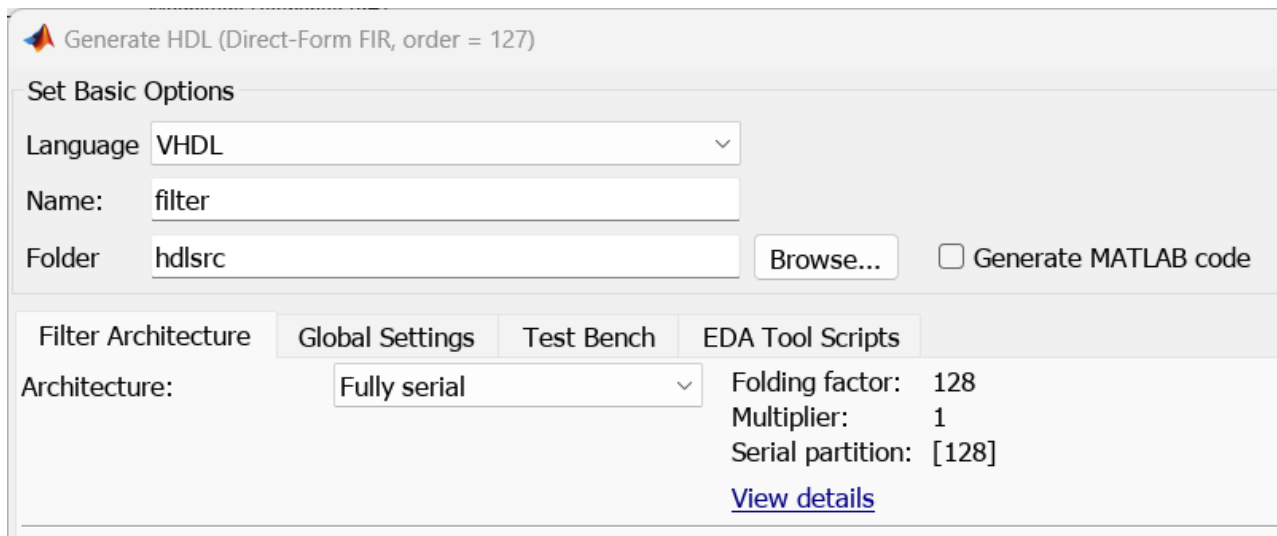


Ilustración 10: Generación del código VHDL desde el fdatool

Como la FPGA a utilizar tiene un máximo de multiplicadores disponibles (12), es necesario dividir el orden de nuestro filtro; cuantos más multiplicadores se usen, más disminuyen los ciclos de trabajo. No se trabaja con arquitectura *fully parallel* puesto que haría el cálculo en un solo ciclo de reloj, usando $n_{\text{multiplicadores}} = \text{orden} = 128$. Esto es imposible porque el

orden del filtro es mayor que 12. Por tanto, se ha escogido trabajar con arquitectura *fully serial* utilizando un solo multiplicador. Dicha arquitectura se resume en la Ilustración 11.



Generate HDL (Direct-Form FIR, order = 127)

Set Basic Options

Language: VHDL

Name: filter

Folder: hdlsrc ☐ Generate MATLAB code

Filter Architecture Global Settings Test Bench EDA Tool Scripts

Architecture: Fully serial

Folding factor: 128

Multiplier: 1

Serial partition: [128]

[View details](#)

Ilustración 11: Configuración *fully serial*

Posteriormente, se da click en “Generate”, y Matlab genera los archivos .vhd de la fuente como del testbench a ser probados en Vivado. Se crea un proyecto, cargando los archivos correspondientes (filter.vhd, filter_tb.vhd, constraints) y habilitando como analógicas las señales input y outputs (filter_in y filter_out) de la simulación.

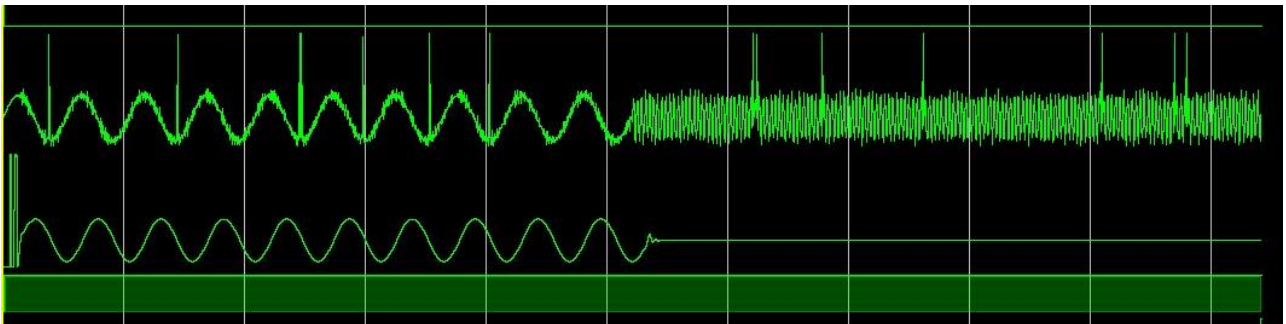


Ilustración 12: Respuesta del filtro en Vivado ante señal con ruido (izquierda) y solo ruido (derecha)



Ilustración 13: Simulación del filtro digital en Vivado

Implementación

Las modificaciones realizadas para utilizar la FPGA son:

- Ancho de palabra del input (ADC) = 12 bits y del input (DAC) = 10 bits.
- Frecuencia de muestreo = 100kHz
 - La frecuencia de corte se escala (y lo mismo con la de stop):

$$f_c^{FPGA} = f_c^{Real} \frac{f_s^{FPGA}}{f_s^{Real}} = 100 * \frac{100000}{10000} = 1000Hz$$

Pruebas - Señales sinusoidales:

Como se puede observar en la Ilustración 14, nuestro filtro deja pasar las señales sinusoidales menores a 1000 Hz, mientras que las mayores a 3000 Hz las desprecia totalmente.

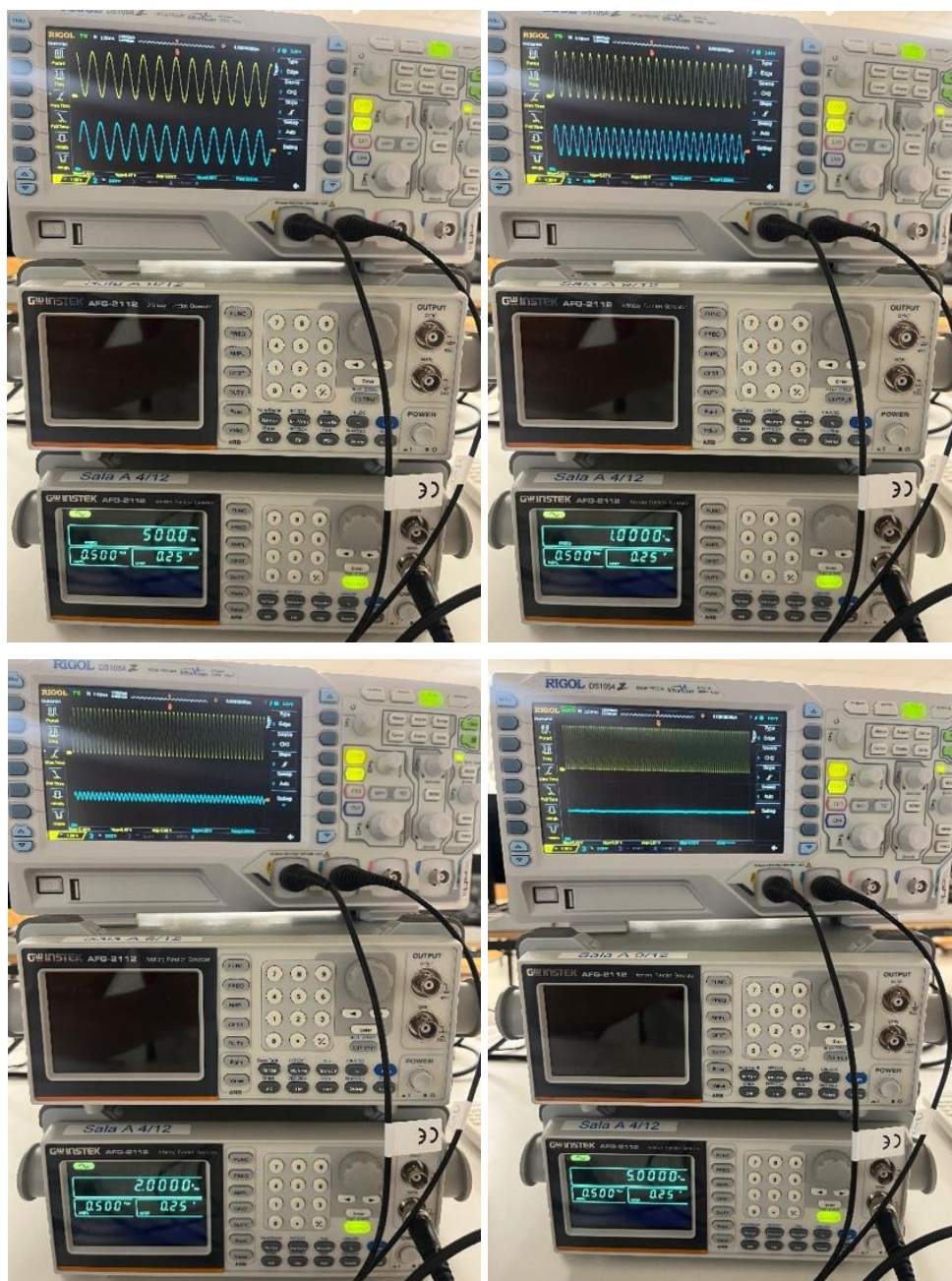


Ilustración 14: Comportamiento del filtro ante señales sinusoidales (de izquierda a derecha) de 500Hz, 1kHz, 2kHz y 5kHz

Pruebas - Señales cuadradas:

La particularidad de las señales cuadradas es que, cuando su frecuencia es suficientemente cercana a la de corte, se produce el efecto “Batman”, tal y como se muestra en las Ilustración 15. En este caso, **como el filtro solo deja pasar muy bajas frecuencias, tan solo se ven 3 armónicos**, y este efecto se ve ligeramente menos acentuado que para otros filtros.

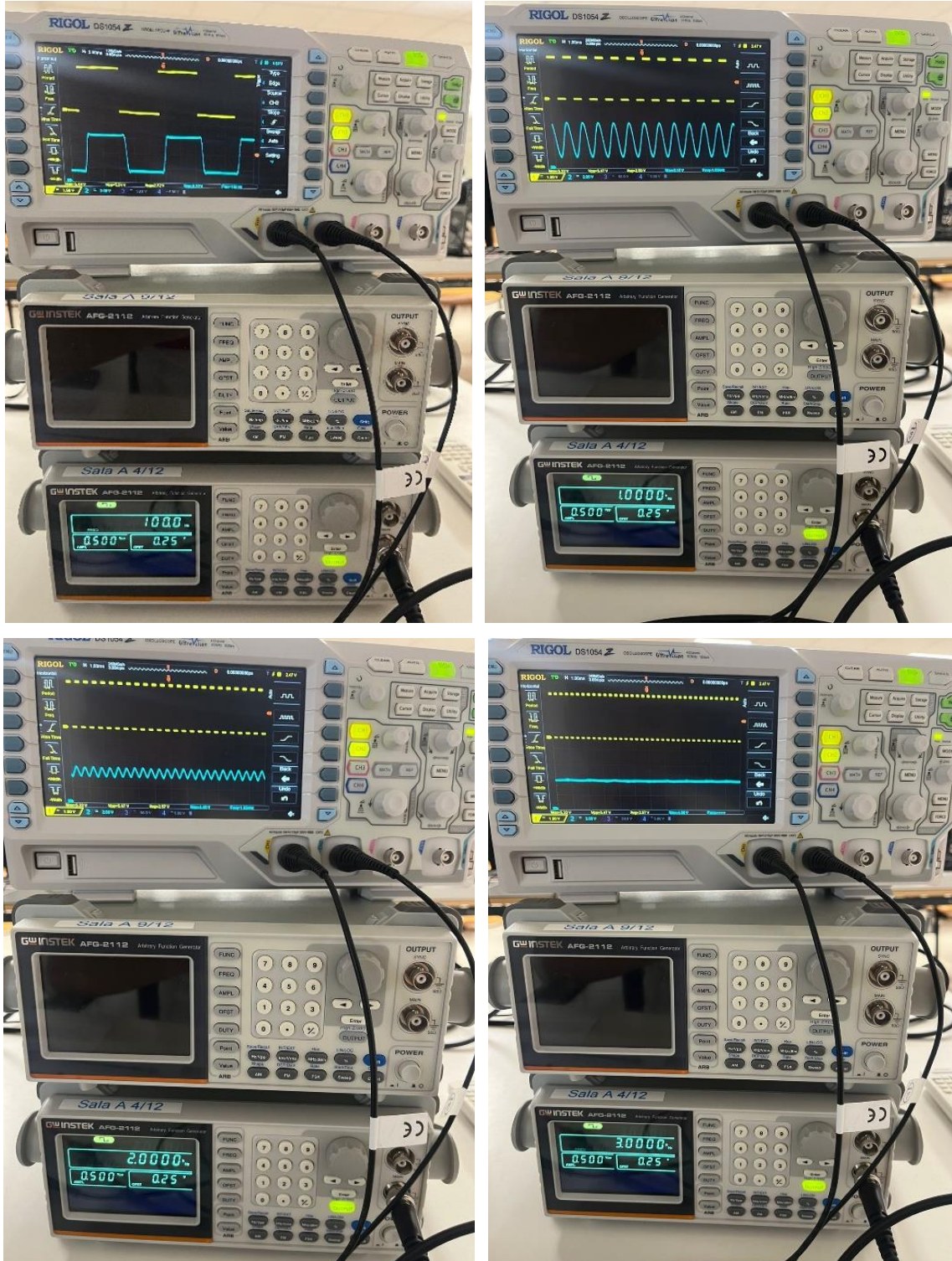


Ilustración 15: Comportamiento del filtro ante señales cuadradas (de izquierda a derecha) de 100Hz, 1kHz, 3kHz y 5kHz

Pruebas - Señal ruidosa:

En la Ilustración 16 se ve claramente como la señal sale (azul) mucho más limpia de lo que entra (morada), reflejando así el comportamiento deseado para la dirección del monoplaza de Formula Student.

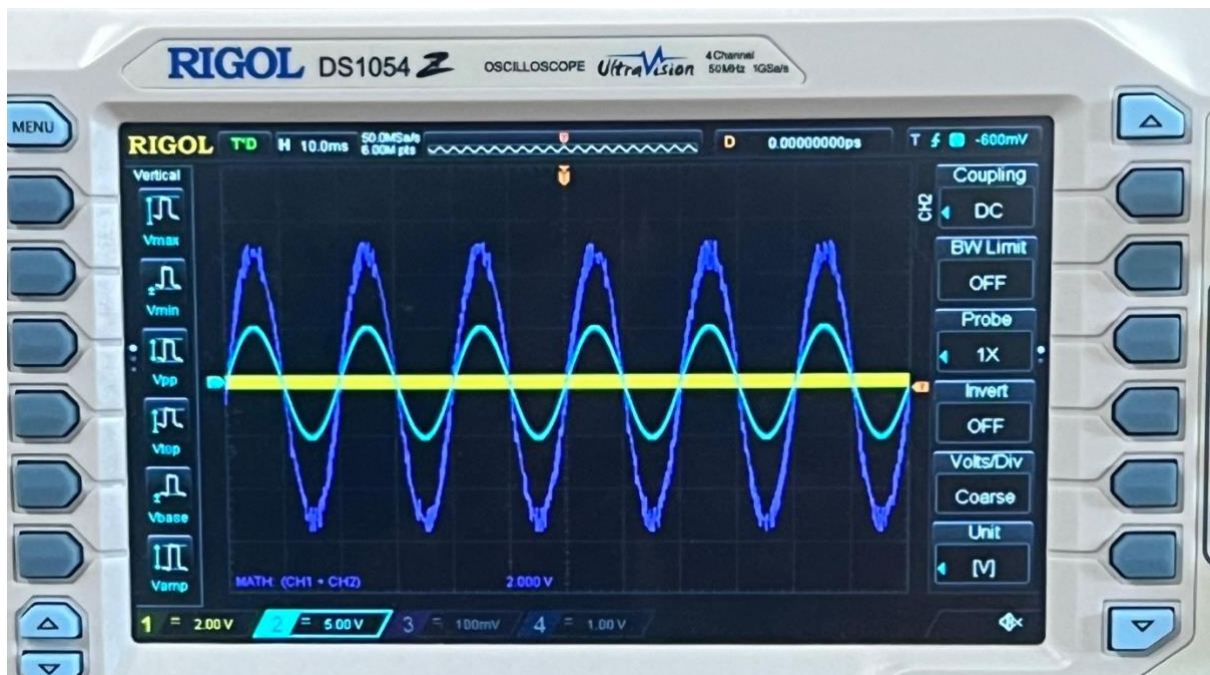


Ilustración 16: Señal onjetivo ruidosa (morada) y filtrada (azul)

Repetición de la señal en la frecuencia de muestreo: $f_s = 100 \text{ kHz}$

Finalmente, en la Ilustración 17 se observa que el filtro es digital, puesto que la señal se repite en frecuencias que son múltiplos de la frecuencia de muestreo, representando las réplicas espectrales de la señal original. El espectro de la señal muestreada se repite cada $N * f_s$.



Ilustración 17: Comportamiento del filtro para señales de 101kHz (de izquierda a derecha) sinusoidal, cuadrada y triangular.